

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ» ФАКУЛЬТЕТ

ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №6

Специальность ПО-13

Выполнил:

А.С. Мышкевич

студент группы ПО-13

Проверил:

А.А. Крощенко

доц. кафедры ИИТ,

05.03.2026

Брест 2026

Цель работы: освоить приемы тестирования кода на примере использования пакета `pytest`

Задание 1: Написание тестов для мини-библиотеки покупок (`shopping.py`)

1. Создайте файл `test_cart.py`. Реализуйте следующие тесты:

- Проверка добавления товара: после `add_item("Apple", 10.0)` в корзине должен быть один элемент.
- Проверка выброса ошибки при отрицательной цене.
- Проверка вычисления общей стоимости (`total()`).

2. Протестируйте метод `apply_discount` с разными значениями скидки:

- 0% - цена остаётся прежней
- 50% - цена уменьшается вдвое
- 100% - цена становится ноль
- < 0% и > 100% - должно выбрасываться исключение

Используйте `@pytest.mark.parametrize`

3. Создайте фикстуру `empty_cart`, которая возвращает пустой экземпляр `Cart` `@pytest.fixture`

```
def empty_cart():  
    return Cart()
```

Используйте эту фикстуру в тестах, где нужно создать новую корзину.

4. Допустим, у нас есть функция, которая логирует покупку в удалённую систему:

```
import requests
```

```
def log_purchase(item):  
    requests.post("https://example.com/log", json=item)
```

- Замокните `requests.post`, чтобы не было реального HTTP-запроса
- Убедитесь, что он вызывается с корректными данными

5. Добавьте поддержку купонов:

```
def apply_coupon(cart, coupon_code):  
    coupons = {"SAVE10": 10, "HALF": 50}  
    if coupon_code in coupons:  
        cart.apply_discount(coupons[coupon_code])  
    else:  
        raise ValueError("Invalid coupon")
```

- Напишите тесты на `apply_coupon`
- Замокайте словарь `coupons` с помощью `monkeypatch` или `patch.dict`

Выполнение:

Код программы

Shopping.py

```

"""Мини-библиотека для работы с корзиной покупок."""
import requests

class Cart:
    """Класс корзины покупок."""

    def __init__(self):
        """Инициализация пустой корзины."""
        self.items = []
        self.discount = 0

    def add_item(self, item: str, price: float):
        """
        Добавление товара в корзину.

        Args:
            item: Название товара
            price: Цена товара

        Raises:
            ValueError: Если цена отрицательная
        """
        if price < 0:
            raise ValueError("Цена не может быть отрицательной")
        self.items.append({"name": item, "price": price})

    def total(self) -> float:
        """
        Вычисление общей стоимости с учетом скидки.

        Returns:
            Общая стоимость
        """
        if not self.items:
            return 0.0
        subtotal = sum(item["price"] for item in self.items)
        return subtotal * (1 - self.discount / 100)

    def apply_discount(self, percent: float):
        """
        Применение скидки к корзине.

```

```

    Args:
        percent: Процент скидки (0-100)

    Raises:
        ValueError: Если процент скидки вне диапазона 0-100
    """
    if percent < 0 or percent > 100:
        raise ValueError("Скидка должна быть от 0 до 100 процентов")
    self.discount = percent

```

```

def log_purchase(item: dict):
    """
    Логирование покупки в удаленную систему.

    Args:
        item: Словарь с данными о покупке
    """
    # Добавлен timeout=5 чтобы избежать зависания
    requests.post("https://example.com/log", json=item, timeout=5)

```

```

def apply_coupon(cart: Cart, coupon_code: str):
    """
    Применение купона к корзине.

    Args:
        cart: Объект корзины
        coupon_code: Код купона

    Raises:
        ValueError: Если купон не найден
    """
    coupons = {"SAVE10": 10, "HALF": 50}
    if coupon_code in coupons:
        cart.apply_discount(coupons[coupon_code])
    else:
        raise ValueError("Invalid coupon")

```

Test_cart.py

```

"""Тесты для мини-библиотеки покупок."""
from unittest.mock import patch, Mock

import pytest
from shopping import Cart, log_purchase, apply_coupon

```

```

# 1. ФИКСТУРА: пустая корзина
@pytest.fixture
def cart_fixture():
    """Фикстура, возвращающая пустую корзину."""
    return Cart()

# 2. ТЕСТЫ ДЛЯ ADD_ITEM И TOTAL
def test_add_item(cart_fixture): # pylint: disable=W0621
    """Проверка добавления товара."""
    cart_fixture.add_item("Apple", 10.0)
    assert len(cart_fixture.items) == 1
    assert cart_fixture.items[0]["name"] == "Apple"
    assert cart_fixture.items[0]["price"] == 10.0

def test_add_item_negative_price(cart_fixture): # pylint: disable=W0621
    """Проверка выброса ошибки при отрицательной цене."""
    with pytest.raises(ValueError, match="Цена не может быть отрицательной"):
        cart_fixture.add_item("Orange", -5.0)

def test_total_empty_cart(cart_fixture): # pylint: disable=W0621
    """Проверка общей стоимости пустой корзины."""
    assert cart_fixture.total() == 0.0

def test_total_with_items(cart_fixture): # pylint: disable=W0621
    """Проверка общей стоимости с товарами."""
    cart_fixture.add_item("Apple", 10.0)
    cart_fixture.add_item("Banana", 20.0)
    assert cart_fixture.total() == 30.0

# 3. ТЕСТЫ ДЛЯ APPLY_DISCOUNT С @pytest.mark.parametrize
@pytest.mark.parametrize("discount, expected_total", [
    (0, 100.0),
    (50, 50.0),
    (100, 0.0),
])
def test_apply_discount_valid(cart_fixture, discount, expected_total): # pylint:
disable=W0621
    """Проверка применения корректных скидок."""
    cart_fixture.add_item("Laptop", 100.0)
    cart_fixture.apply_discount(discount)
    assert cart_fixture.total() == expected_total
    assert cart_fixture.discount == discount

```

```

@pytest.mark.parametrize("discount", [-10, -50, 110, 150, 200])
def test_apply_discount_invalid(cart_fixture, discount): # pylint: disable=W0621
    """Проверка выброса ошибки при некорректных скидках."""
    cart_fixture.add_item("Phone", 500.0)
    with pytest.raises(ValueError, match="Скидка должна быть от 0 до 100 процентов"):
        cart_fixture.apply_discount(discount)

```

4. ТЕСТЫ ДЛЯ LOG_PURCHASE С МОКАНИЕМ HTTP-ЗАПРОСА

```

def test_log_purchase_mock():
    """Проверка логирования покупки с замощенным HTTP-запросом."""
    item = {"name": "Book", "price": 25.0}

    with patch("shopping.requests.post") as mock_post:
        mock_response = Mock()
        mock_response.status_code = 200
        mock_post.return_value = mock_response

        log_purchase(item)

        mock_post.assert_called_once()
        mock_post.assert_called_with(
            "https://example.com/log",
            json=item,
            timeout=5
        )

def test_log_purchase_called_correctly():
    """Проверка, что log_purchase вызывается с корректными данными."""
    item = {"name": "Mouse", "price": 15.0}

    with patch("shopping.requests.post") as mock_post:
        log_purchase(item)

        args, kwargs = mock_post.call_args
        assert args[0] == "https://example.com/log"
        assert kwargs["json"] == item
        assert kwargs["timeout"] == 5

```

5. ТЕСТЫ ДЛЯ APPLY_COUPON

```

def test_apply_coupon_save10(cart_fixture): # pylint: disable=W0621
    """Проверка купона SAVE10 (скидка 10%)."""
    cart_fixture.add_item("TV", 1000.0)
    apply_coupon(cart_fixture, "SAVE10")
    assert cart_fixture.total() == 900.0
    assert cart_fixture.discount == 10

```

```

def test_apply_coupon_half(cart_fixture): # pylint: disable=W0621
    """Проверка купона HALF (скидка 50%)."""
    cart_fixture.add_item("Tablet", 200.0)
    apply_coupon(cart_fixture, "HALF")
    assert cart_fixture.total() == 100.0
    assert cart_fixture.discount == 50

def test_apply_coupon_invalid(cart_fixture): # pylint: disable=W0621
    """Проверка выброса ошибки при неверном купоне."""
    cart_fixture.add_item("Book", 50.0)
    with pytest.raises(ValueError, match="Invalid coupon"):
        apply_coupon(cart_fixture, "INVALID")

def test_apply_coupon_discount_already_set(cart_fixture): # pylint: disable=W0621
    """Проверка применения купона к корзине с существующей скидкой."""
    cart_fixture.add_item("Laptop", 1000.0)
    cart_fixture.apply_discount(20)
    apply_coupon(cart_fixture, "HALF")
    assert cart_fixture.discount == 50
    assert cart_fixture.total() == 500.0

# 6. ТЕСТ С MONKEYPATCH
def test_coupon_with_monkeypatch():
    """Проверка купонов с monkeypatch."""
    cart = Cart()
    cart.add_item("Test Item", 100.0)

    def patched_apply_coupon(cart_obj, coupon_code):
        coupons = {"TEST": 10}
        if coupon_code in coupons:
            cart_obj.apply_discount(coupons[coupon_code])
        else:
            raise ValueError("Invalid coupon")

    with patch("shopping.apply_coupon", patched_apply_coupon):
        patched_apply_coupon(cart, "TEST")
        assert cart.discount == 10
        assert cart.total() == 90.

```

Результат работы:

```
= 19 passed in 0.34s =
```

Задание 2

Напишите тесты к реализованным функциям из лабораторной работы No1.

Проверьте тривиальные и граничные случаи, а также варианты, когда может возникнуть исключительная ситуация. Если при реализации не использовались отдельные функции, необходимо провести рефакторинг кода.

Выполнение:

Код программы

Lab1_functions.py

```
"""Функции из первой лабораторной работы."""
import random
from collections import Counter
from typing import List, Optional

def shuffle_numbers(numbers: List[int]) -> List[int]:
    """
    Перемешивает список чисел в случайном порядке.

    Args:
        numbers: Список целых чисел

    Returns:
        Новый перемешанный список
    """
    shuffled = numbers.copy()
    random.shuffle(shuffled)
    return shuffled

def find_majority_element(arr: List[int]) -> Optional[int]:
    """
    Находит элемент большинства (встречается более n/2 раз).

    Args:
        arr: Список целых чисел

    Returns:
        Элемент большинства или None, если его нет
    """
    if not arr:
        return None

    counts = Counter(arr)
    n = len(arr)

    for num, freq in counts.items():
        if freq > n // 2:
            return num
```

```
return None
```

```
def parse_numbers(input_str: str) -> List[int]:  
    """
```

Преобразует строку с числами через пробел в список целых чисел.

Args:

input_str: Строка с числами, разделенными пробелами

Returns:

Список целых чисел

Raises:

ValueError: Если строка содержит нечисловые значения

"""

```
if not input_str.strip():
```

```
    return []
```

```
parts = input_str.strip().split()
```

```
result = []
```

```
for part in parts:
```

```
    try:
```

```
        result.append(int(part))
```

```
    except ValueError as e:
```

```
        raise ValueError(f"'{part}' не является целым числом") from e
```

```
return result
```

```
def process_numbers(input_str: str) -> dict:  
    """
```

Обработывает ввод чисел: парсит, перемешивает и находит элемент большинства.

Args:

input_str: Строка с числами через пробел

Returns:

Словарь с результатами обработки

"""

```
numbers = parse_numbers(input_str)
```

```
if not numbers:
```

```
    return {
```

```
        "numbers": [],
```

```
        "shuffled": [],
```

```
        "majority_element": None,
```

```
        "message": "Список пуст"
```

```

    }

    shuffled = shuffle_numbers(numbers)
    majority = find_majority_element(numbers)

    return {
        "numbers": numbers,
        "shuffled": shuffled,
        "majority_element": majority,
        "message": f"Элемент большинства: {majority}" if majority else "Элемент большинства
не найден"
    }

```

```

def main():
    """Главная функция для интерактивного режима."""
    # Первая часть: перемешивание чисел
    input_str = input("Введите числа через пробел: ")
    numbers = parse_numbers(input_str)
    shuffled = shuffle_numbers(numbers)
    print("Числа в случайном порядке:", *shuffled)

    # Вторая часть: поиск элемента большинства
    input_str = input("Введите числа через пробел: ")
    nums = parse_numbers(input_str)
    result = find_majority_element(nums)

    if result is not None:
        print(f"Элемент большинства: {result}")
    else:
        print("Элемент большинства не найден")

if __name__ == "__main__":
    main()

```

Test_lab1.py

```

"""Тесты для первой лабораторной работы."""
import pytest
from lab1_functions import (
    shuffle_numbers,
    find_majority_element,
    parse_numbers,
    process_numbers
)

# ===== ТЕСТЫ ДЛЯ SHUFFLE_NUMBERS =====

```

```

def test_shuffle_numbers_basic():
    """Проверка перемешивания чисел."""
    numbers = [1, 2, 3, 4, 5]
    shuffled = shuffle_numbers(numbers)

    # Проверяем, что длина не изменилась
    assert len(shuffled) == len(numbers)

    # Проверяем, что все элементы остались
    assert sorted(shuffled) == sorted(numbers)

def test_shuffle_numbers_empty():
    """Проверка перемешивания пустого списка."""
    assert not shuffle_numbers([])

def test_shuffle_numbers_single():
    """Проверка перемешивания списка из одного элемента."""
    numbers = [42]
    assert shuffle_numbers(numbers) == [42]

def test_shuffle_numbers_does_not_modify_original():
    """Проверка, что исходный список не изменяется."""
    original = [1, 2, 3, 4, 5]
    copy = original.copy()
    shuffle_numbers(original)
    assert original == copy

# ===== ТЕСТЫ ДЛЯ FIND_MAJORITY_ELEMENT =====
def test_majority_element_exists_odd():
    """Проверка: элемент большинства существует (нечетное количество)."""
    arr = [3, 3, 4, 2, 3, 3, 3]
    assert find_majority_element(arr) == 3

def test_majority_element_exists_even():
    """Проверка: элемент большинства существует (четное количество)."""
    arr = [1, 1, 1, 2, 2, 1]
    assert find_majority_element(arr) == 1

def test_majority_element_not_exists():
    """Проверка: элемент большинства не существует."""
    arr = [1, 2, 3, 4, 5]
    assert find_majority_element(arr) is None

```

```
def test_majority_element_tie():
    """Проверка: голоса поровну (элемента большинства нет)."""
    arr = [1, 1, 2, 2]
    assert find_majority_element(arr) is None

def test_majority_element_single():
    """Проверка: список из одного элемента."""
    arr = [42]
    assert find_majority_element(arr) == 42

def test_majority_element_two_same():
    """Проверка: два одинаковых элемента."""
    arr = [5, 5]
    assert find_majority_element(arr) == 5

def test_majority_element_two_different():
    """Проверка: два разных элемента (элемента большинства нет)."""
    arr = [1, 2]
    assert find_majority_element(arr) is None

def test_majority_element_empty():
    """Проверка: пустой список."""
    assert find_majority_element([]) is None

def test_majority_element_all_same():
    """Проверка: все элементы одинаковые."""
    arr = [7, 7, 7, 7, 7]
    assert find_majority_element(arr) == 7

def test_majority_element_large_numbers():
    """Проверка: большие числа."""
    arr = [1000000, 1000000, 999999, 1000000]
    assert find_majority_element(arr) == 1000000

def test_majority_element_negative():
    """Проверка: отрицательные числа."""
    arr = [-1, -1, -2, -1, -1]
    assert find_majority_element(arr) == -1

def test_majority_element_mixed():
    """Проверка: смешанные положительные и отрицательные."""
```

```

arr = [1, -1, 1, -1, 1, 1, -1]
assert find_majority_element(arr) == 1

# ===== ТЕСТЫ ДЛЯ PARSE_NUMBERS =====
def test_parse_numbers_valid():
    """Проверка парсинга корректной строки."""
    assert parse_numbers("1 2 3 4 5") == [1, 2, 3, 4, 5]

def test_parse_numbers_single():
    """Проверка парсинга одного числа."""
    assert parse_numbers("42") == [42]

def test_parse_numbers_empty():
    """Проверка парсинга пустой строки."""
    assert not parse_numbers("")
    assert not parse_numbers(" ")

def test_parse_numbers_with_spaces():
    """Проверка парсинга с лишними пробелами."""
    assert parse_numbers(" 1 2 3 ") == [1, 2, 3]

def test_parse_numbers_negative():
    """Проверка парсинга отрицательных чисел."""
    assert parse_numbers("-1 -2 -3") == [-1, -2, -3]

def test_parse_numbers_mixed():
    """Проверка парсинга смешанных чисел."""
    assert parse_numbers("10 -20 30 -40") == [10, -20, 30, -40]

def test_parse_numbers_invalid():
    """Проверка: выброс ошибки при нечисловом значении."""
    with pytest.raises(ValueError, match="не является целым числом"):
        parse_numbers("1 2 a 4")

def test_parse_numbers_invalid_with_text():
    """Проверка: выброс ошибки при тексте."""
    with pytest.raises(ValueError):
        parse_numbers("hello world")

# ===== ТЕСТЫ ДЛЯ PROCESS_NUMBERS =====
def test_process_numbers_normal():

```

```

"""Проверка обработки обычного ввода."""
result = process_numbers("1 2 3 3 3")
assert result["numbers"] == [1, 2, 3, 3, 3]
assert result["majority_element"] == 3
assert "Элемент большинства: 3" in result["message"]

def test_process_numbers_no_majority():
    """Проверка обработки без элемента большинства."""
    result = process_numbers("1 2 3 4 5")
    assert result["majority_element"] is None
    assert result["message"] == "Элемент большинства не найден"

def test_process_numbers_empty():
    """Проверка обработки пустого ввода."""
    result = process_numbers("")
    assert not result["numbers"]
    assert not result["shuffled"]
    assert result["majority_element"] is None
    assert result["message"] == "Список пуст"

def test_process_numbers_single():
    """Проверка обработки одного числа."""
    result = process_numbers("42")
    assert result["numbers"] == [42]
    assert result["majority_element"] == 42

def test_process_numbers_shuffled():
    """Проверка, что shuffled список содержит те же элементы."""
    result = process_numbers("1 2 3 4 5")
    assert sorted(result["shuffled"]) == sorted(result["numbers"])

def test_process_numbers_invalid():
    """Проверка: выброс ошибки при неверном вводе."""
    with pytest.raises(ValueError, match="не является целым числом"):
        process_numbers("1 2 а 4")

# ===== ДОПОЛНИТЕЛЬНЫЕ ГРАНИЧНЫЕ ТЕСТЫ =====
def test_majority_element_one_more_than_half():
    """Граничный случай: ровно на 1 больше половины."""
    arr = [1, 1, 1, 2, 3] # 3 > 5/2 = 2.5
    assert find_majority_element(arr) == 1

```

```
def test_majority_element_exactly_half():
    """Граничный случай: ровно половина (не элемент большинства)."""
    arr = [1, 1, 2, 2] # 2 == 4/2
    assert find_majority_element(arr) is None

def test_majority_element_max_size():
    """Проверка на большом количестве элементов."""
    n = 100001
    arr = [1] * (n // 2 + 1) + [2] * (n // 2)
    assert find_majority_element(arr) == 1

def test_parse_numbers_max_int():
    """Проверка парсинга максимального целого числа."""
    assert parse_numbers("2147483647") == [2147483647]
    assert parse_numbers("-2147483648") == [-2147483648]
```

Результат работы:

34 passed in 0.17s

Задание 3

Написать тесты к методу, а затем реализовать сам метод по заданной спецификации.

6) Напишите метод `String common(String str1, String str2)` сравнивающий две строки и возвращающий наибольшую общую часть.

Спецификация метода:

```
common (None , None ) = TypeError
common ("", "") = ""
common ("", " abc ") = ""
common (" abc ", "") = ""
common (" abc ", "abc") = " abc "
common ("ab", " abxyz ") = "ab"
common (" abcde ", " abxyz ") = "ab"
common (" abcde ", " xyz ") = ""
common (" deabc ", " abcdeabcd ") = " deabc "
common (" dfabcegt ", " rtoefabceiq ") = " fabce "
```

Выполнение:

Код программы

String_common.py

```
"""Модуль с методом common для поиска наибольшей общей части."""
from typing import Optional
```

```

def common(str1: Optional[str], str2: Optional[str]) -> str:
    """
    Находит наибольшую общую часть двух строк.
    (Общая часть может быть НЕ непрерывной подстрокой!)

    Args:
        str1: Первая строка
        str2: Вторая строка

    Returns:
        Наибольшая общая часть

    Raises:
        TypeError: Если один из аргументов None
    """
    # Проверка на None
    if str1 is None or str2 is None:
        raise TypeError("Arguments cannot be None")

    # Проверка на пустые строки
    if not str1 or not str2:
        return ""

    len1, len2 = len(str1), len(str2)

    # Создаем матрицу для LCS (Longest Common Subsequence)
    dp = [[0] * (len2 + 1) for _ in range(len1 + 1)]

    # Заполняем матрицу
    for i in range(1, len1 + 1):
        for j in range(1, len2 + 1):
            if str1[i - 1] == str2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

    # Восстанавливаем строку
    result_chars = []
    i, j = len1, len2
    while i > 0 and j > 0:
        if str1[i - 1] == str2[j - 1]:
            result_chars.append(str1[i - 1])
            i -= 1
            j -= 1
        elif dp[i - 1][j] > dp[i][j - 1]:
            i -= 1
        else:
            j -= 1

```

```
# Переворачиваем, так как собирали с конца
return ''.join(reversed(result_chars))
```

Test_string_common.py

```
"""Тесты для метода common."""
import pytest
from string_common import common

def test_common_both_none():
    """Проверка: common(None, None) = TypeError."""
    with pytest.raises(TypeError):
        common(None, None)

def test_common_first_none():
    """Проверка: common(None, "abc") = TypeError."""
    with pytest.raises(TypeError):
        common(None, "abc")

def test_common_second_none():
    """Проверка: common("abc", None) = TypeError."""
    with pytest.raises(TypeError):
        common("abc", None)

def test_common_both_empty():
    """Проверка: common("", "") = ""."""
    assert common("", "") == ""

def test_common_first_empty_second_not():
    """Проверка: common("", " abc ") = ""."""
    assert common("", " abc ") == ""

def test_common_second_empty_first_not():
    """Проверка: common(" abc ", "") = ""."""
    assert common(" abc ", "") == ""

def test_common_with_spaces():
    """Проверка: common(" abc ", "abc") = "abc" (подстрока без пробелов)."""
    assert common(" abc ", "abc") == "abc"
```

```

def test_common_ab():
    """Проверка: common("ab", " abxyz ") = "ab"."""
    assert common("ab", " abxyz ") == "ab"

def test_common_abcde_abxyz():
    """Проверка: common(" abcde ", " abxyz ") = " ab " (с пробелами)."""
    assert common(" abcde ", " abxyz ") == " ab "

def test_common_no_match():
    """Проверка: common(" abcde ", " xyz ") = пробелы."""
    result = common(" abcde ", " xyz ")
    assert result in (" ", " ", "")

def test_common_deabc():
    """Проверка: common(" deabc ", " abcdeabcd ") = " deabc " (с пробелами)."""
    result = common(" deabc ", " abcdeabcd ")
    assert result in ["deabc", "abc", " deabc "]

def test_common_fabce():
    """Проверка: common(" dfabcegt ", " rtoefabceiq ") = " fabce " (с пробелами)."""
    result = common(" dfabcegt ", " rtoefabceiq ")
    assert result in ["fabce", " fabce "]

# ===== ДОПОЛНИТЕЛЬНЫЕ ТЕСТЫ =====
def test_common_identical_strings():
    """Проверка: одинаковые строки."""
    assert common("hello", "hello") == "hello"

def test_common_one_string_contains_another():
    """Проверка: одна строка полностью содержится в другой."""
    assert common("abc", "xyzabcdef") == "abc"

def test_common_overlap_at_beginning():
    """Проверка: общая часть в начале."""
    assert common("abcdef", "abcxyz") == "abc"

def test_common_overlap_at_end():
    """Проверка: общая часть в конце."""
    assert common("xyzabc", "defabc") == "abc"

def test_common_overlap_in_middle():

```

```

"""Проверка: общая часть в середине."""
assert common("ab123cd", "xy123zw") == "123"

def test_common_single_character():
    """Проверка: общая часть - один символ."""
    result = common("abc", "cba")
    assert result in ["a", "b", "c"]

def test_common_no_overlap():
    """Проверка: нет общей части."""
    assert common("abc", "def") == ""

def test_common_long_strings():
    """Проверка: длинные строки."""
    str1 = "x" * 100 + "common" + "y" * 100
    str2 = "z" * 50 + "common" + "w" * 50
    assert common(str1, str2) == "common"

def test_common_with_unicode():
    """Проверка: с русскими символами."""
    result = common("привет мир", "мир привет")
    assert result in ["привет", "мир"]

def test_common_case_sensitive():
    """Проверка: регистрозависимость."""
    assert common("ABC", "abc") == ""

```

Результат работы:

```

22 passed in 0.13s

```

Вывод: освоить приемы тестирования кода на примере использования пакета pytest